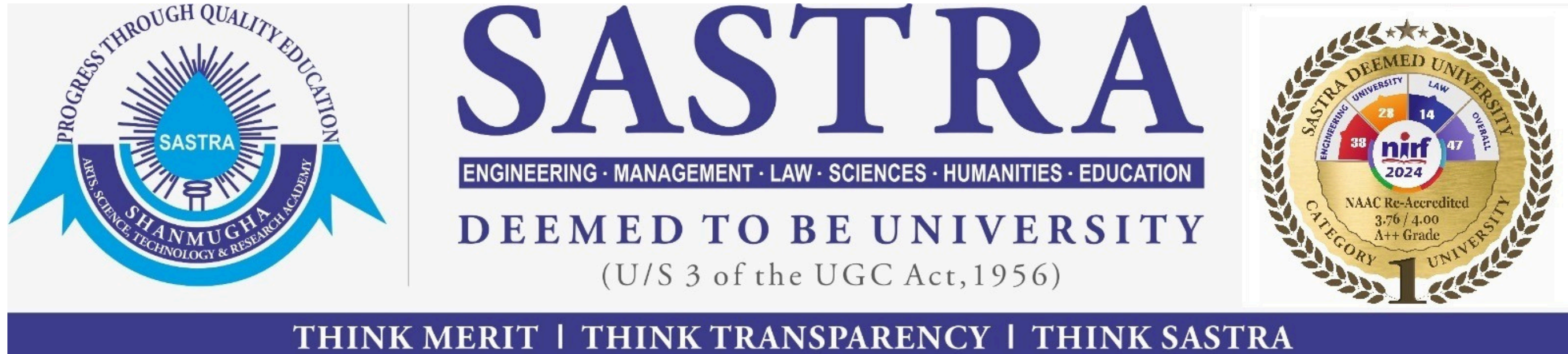




Implementing SPECK-R Cryptographic Algorithm for Secure Communication

126157020 Jagannivasan P
126157035 Muhammad Nadeem T
126157005 Akash U

Guided By,
Dr.R.Alageswaran
Associate Dean-Student welfare & Professor , School of Computing

LAYOUT OF THIS PRESENTATION

- Abstract
- Introduction
- Objective
- Base Paper
- Problem Statement
- Literature survey
- Proposed technique
- Modules
- Conclusion
- References

ABSTRACT

- In modern IoT-based healthcare applications, such as pacemakers, insulin injectors, actuators, sensors, and RFID tags, secure communication is essential to ensure the integrity and confidentiality of sensitive data.
- These applications often rely on resource-constrained devices, which pose challenges when implementing traditional cryptographic algorithms.
- This mini-project focuses on applying Lightweight Cryptography (LWC) using the SPECK-R algorithm to address these security needs in IoT environments.
- Using ESP32 microcontrollers as transmitters and receivers, the system encrypts and decrypts frequency-based signals, ensuring secure communication between devices.
- In such environments, where attackers can eavesdrop, replay, or alter messages over public channels, implementing SPECK-R provides an efficient and optimized cryptographic solution.
- SPECK-R's lightweight design ensures minimal impact on processing power and memory, making it an ideal choice for IoT devices in healthcare and other resource-constrained applications, enhancing security without compromising performance.

INTRODUCTION

- In the rapidly evolving digital world, IoT devices generate and process vast amounts of sensitive data. Ensuring data protection is crucial to prevent unauthorized access, data breaches, and cyber threats.
- The fundamental principles of Information Security—Confidentiality, Integrity, and Availability (CIA)—are critical in IoT networks.
- The resource constraints of IoT devices make it challenging to implement traditional cryptographic solutions (like AES, RSA) without compromising performance and efficiency.
- To address these challenges, Lightweight Cryptography (LWC) is designed to provide robust security with minimal computational overhead.
- By optimizing encryption algorithms for power, memory, and processing efficiency, LWC ensures secure communication in resource-constrained IoT environments.

OBJECTIVE

- Design and implement a secure communication system to enhance the security of IoT networks. The framework addresses the resource constraints of IoT devices across various applications.
- Implementing SPECK-R Lightweight Cryptographic Algorithm for communication between ESP32 microcontrollers.

BASE PAPER



Sleem, L., Couturier, R. Speck-R: An ultra light-weight cryptographic scheme for Internet of Things.
Multimed Tools Appl 80, 17067–17102 (2021).
<https://doi.org/10.1007/s11042-020-09625-8>

PROBLEM STATEMENT

- To implement SPECK-R for minimizing computational overhead, power consumption, and latency while ensuring robust encryption for sensitive data transmission in IoT environments like healthcare, industrial automation, smart homes, and wearable devices.

LITERATURE SURVEY

AUTHOR	TITLE	APPROACH DESCRIPTION AND ACHIEVEMENTS	LIMITATIONS/OBSERVATION	YEAR
Kerry A. McKay Larry Bassham Meltem Sönmez Turan Nicky Mouha	Report on Lightweight Cryptography	NIST held public workshops and collected feedback to shape lightweight cryptography requirements, Development of profile templates to define physical, performance, and security characteristics for different cryptographic application.	Lightweight cryptography must balance security strength with constrained resource availability, different environments require tailored cryptographic solutions, reinforcing the need for adaptable algorithms	2017
Levent Ertaul, Arpit Chauhan	IoT Security: Implementation of Xtea,Simon/Speck Lightweight Block Ciphers	Implemented Xtea, Simon, and Speck lightweight encryption algorithms on an ESP8266 microcontroller to enhance IoT security, End-to-end encryption was used to ensure data confidentiality throughout transmission	Simon/Speck offers strong security but requires more processing time and power compared to Xtea.	2023

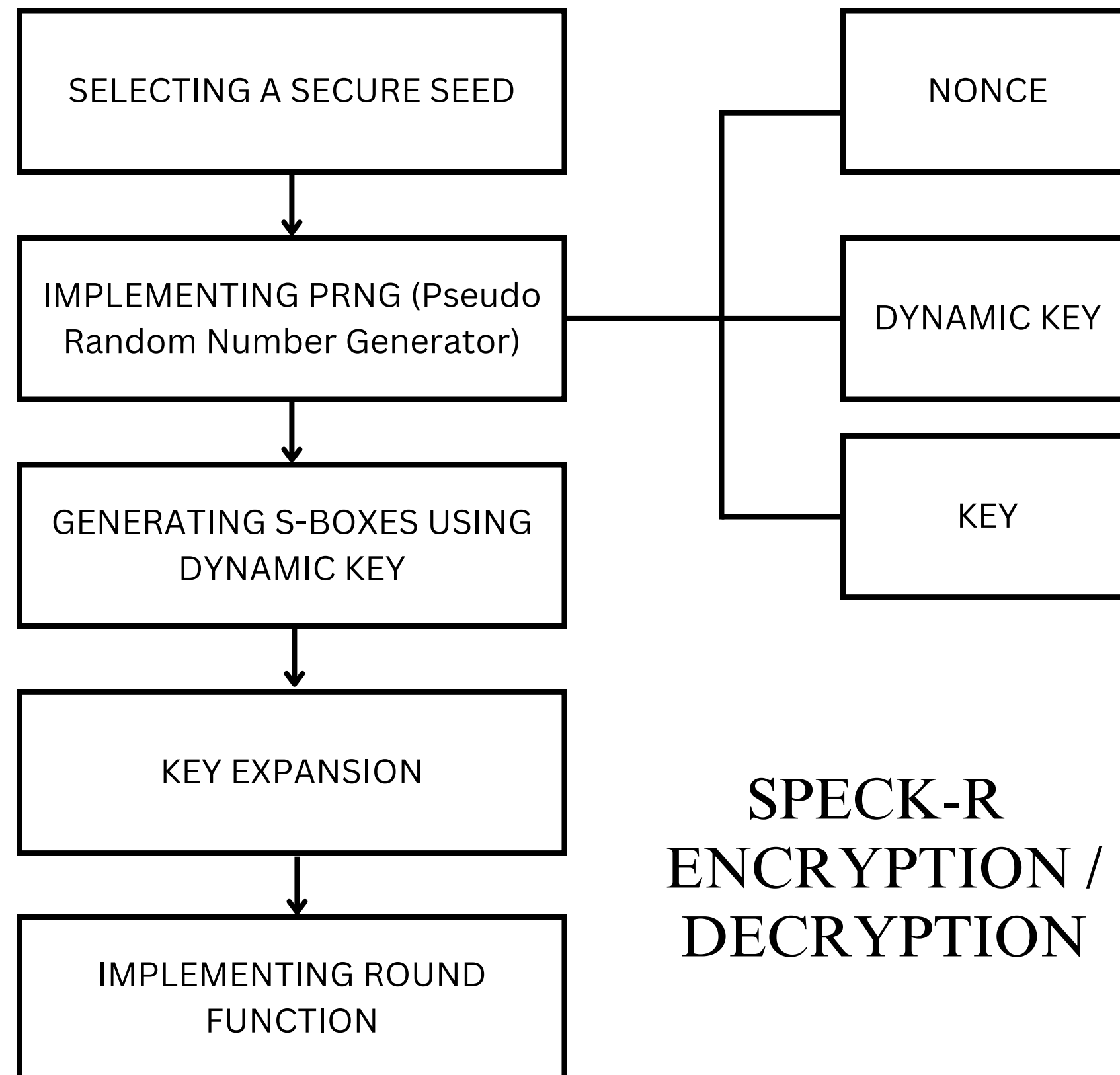
LITERATURE SURVEY

AUTHOR	TITLE	APPROACH DESCRIPTION AND ACHIEVEMENTS	LIMITATIONS/OBSERVATION	YEAR
Guy L. Steele Jr, Doug Lea, Christine H. Flood	Fast Splittable Pseudorandom Number Generators	Derived SPLITMIX from DOTMIX, refining it with engineering improvements for higher speed and better randomness, Developed SPLITMIX, a fast splittable pseudorandom number generator (PRNG), Implemented a splittable PRNG model to enhance parallel computing efficiency without requiring locks or synchronization	Compared to older PRNGs, SPLITMIX offers better statistical properties and avoids common issues like short periods and thread contention	2014
Qian Yu and Chang N. Zhang	RC4 State and Its Applications	Introduced an RC4 state-based hash function for efficient Message Authentication Code (MAC) generation, suitable for resource-constrained environments, Developed an RC4 state-based security protocol that enhances data confidentiality, authentication, integrity, and freshness with minimal overhead.	The proposed approach requires half the execution time compared to WEP and RC5-based protocols, making it twice as efficient.	2011

LITERATURE SURVEY

AUTHOR	TITLE	APPROACH DESCRIPTION AND ACHIEVEMENTS	LIMITATIONS/ OBSERVATION	YEAR
Lama Sleem, Raphael Couturier	Speck-R: An ultra light-weight cryptographic scheme for Internet of Things	Combined ARX (Addition/Rotation/XOR) operations with a dynamic substitution layer, strengthening resistance against attacks. Developed Speck-R, an ultra-lightweight cryptographic algorithm based on Speck, designed for IoT security.	Speck-R achieves a balance between fast execution and strong security, making it ideal for IoT applications	2020
Indu Radhakrishnan, Shruti Jadonand, Prasad B. Honnavalli	Efficiency and Security Evaluation of Lightweight Cryptographic Algorithms for Resource-Constrained IoT Devices	Evaluated the performance of three lightweight cryptographic algorithms—AES-128, SPECK, and ASCON—on resource-constrained IoT devices, Evaluated metrics such as encryption/decryption speed, memory footprint, and power consumption to determine the most efficient algorithm	SPECK exhibited the best performance in terms of throughput and latency, but it has moderate security when it is compared with AES-128 and ASCON.	2024

PROPOSED TECHNIQUE



Requirement :

Sender and receiver must have same seed.

(Implementing symmetric key cryptography).

MODULES

Module 1. Input and Pseudo Random Number Generator Module :

Plaintext transformation : Converting the Plaintext into Blocks

- Submodule 1: Input : Seed.
- Submodule 2: Output : Nonce, Key, Dynamic Key.
- Submodule 3: Implementation of PRNG using SplitMix64.

Module 2. Dynamic Substitution Layer Module :

- Submodule 1: Input as Dynamic Key.
- Submodule 2: Implementation using KSA for RC4.

Module 3. Key Expansion Module : Expanding the key for Round Function.

Module 4. Encryption/Decryption Module : Implementing the Round Function.

MODULE 1

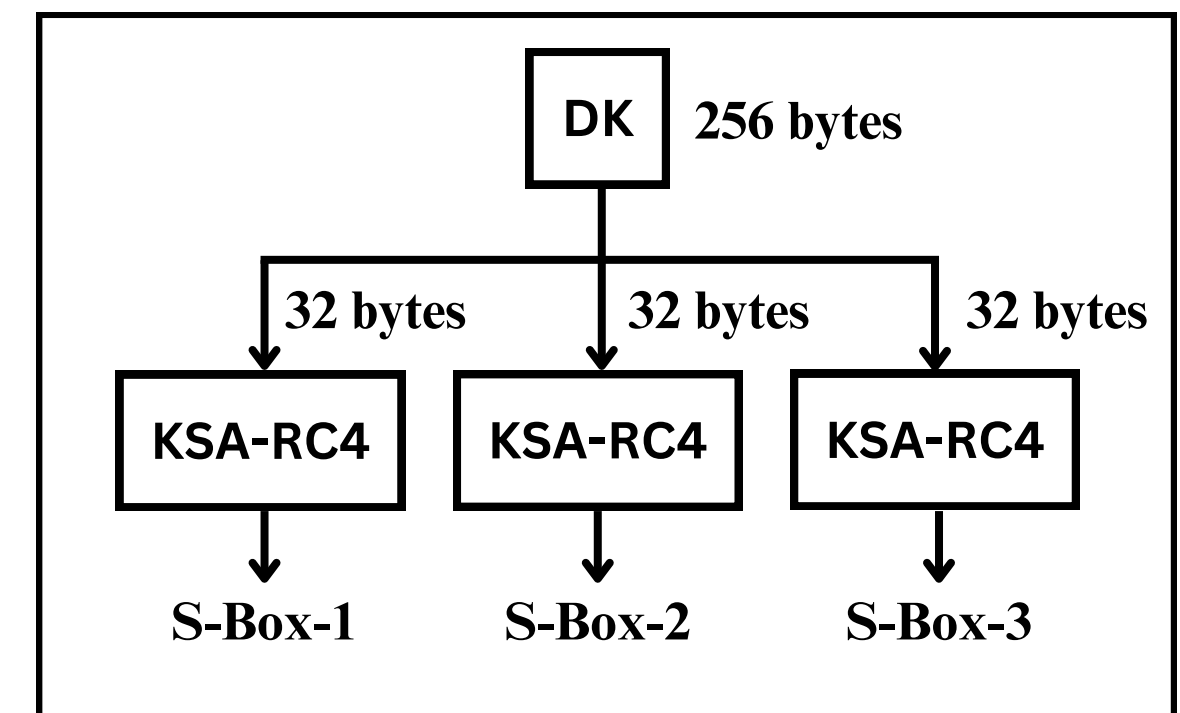
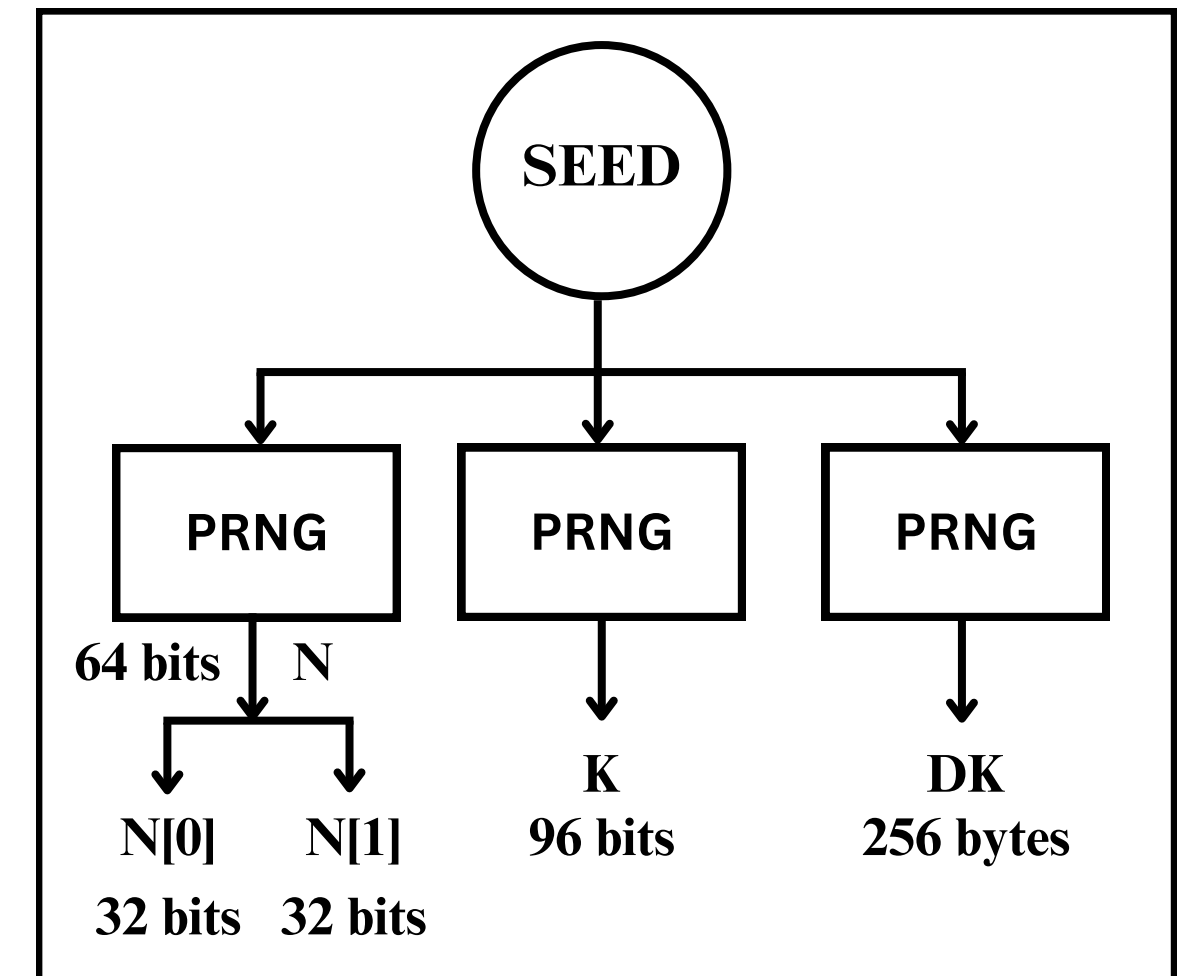
Input and Pseudo Random Number Generator Module

SEED

A seed is an initial value used in algorithms to generate random numbers or to initialize processes. This ensure randomness and uniqueness in outputs.

NONCE, KEY & DYNAMIC KEY

- A **Number Used Once**, is used to ensure that each session is unique and to prevent replay attacks. Here It is generated using SplitMix64 PRNG [64-bits].
- A **Symmetric Key** [96-bits] is generated using a PRNG and expanded for each of the 7 rounds in the round function.
- A **Dynamic Key** [256 bytes] is used to generate dynamic substitution tables (here 3 Sboxes) that change with each encryption session. This enhances security by making the cipher more resistant to attacks like differential and linear cryptanalysis.



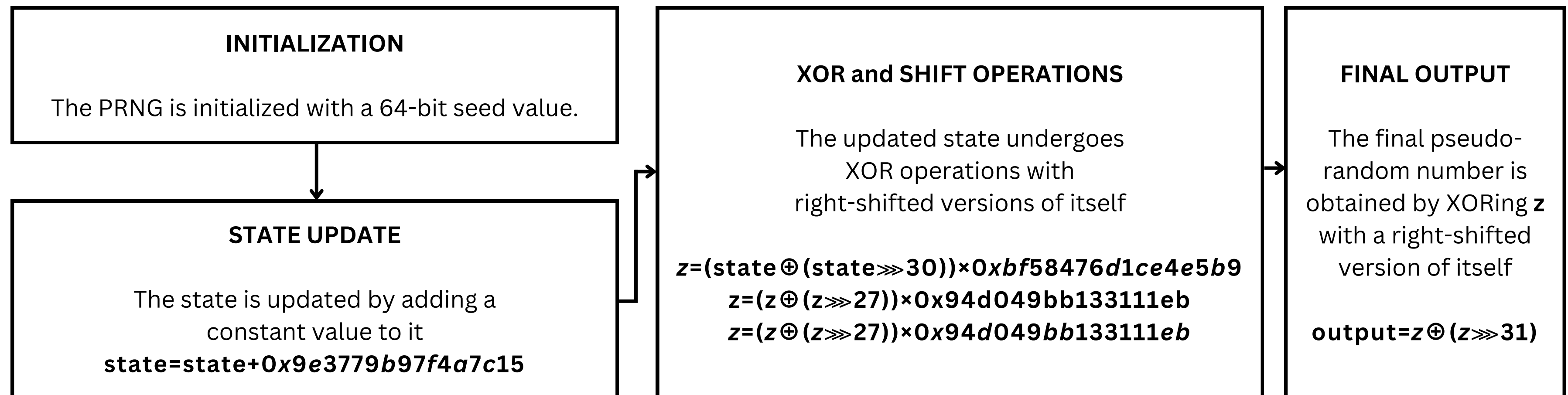
MODULE 1 (cond...)

Input and Pseudo Random Number Generator Module

Implementation of PRNG using SplitMix64

SplitMix64 is chosen for its speed and simplicity. It is seeded with a 64-bit integer and generates random numbers through a series of bitwise operations and multiplications.

Algorithm : SplitMix64 PRNG



MODULE 2

Dynamic Substitution Layer Module

The Dynamic Substitution Layer is used to enhance security by introducing non-linearity and randomness into the encryption process. The state of the substitution tables depends on the dynamic key, which means any change in the key results in a different set of substitution tables.

Algorithm : KSA for RC4

```
procedure RC4_KSA( $K = \{k_1, k_2, \dots, k_L\}, L$ )
  for  $i \leftarrow 0$  to 255 do
     $S[i] \leftarrow i$ 
  end for
   $j \leftarrow 0$ 
  for  $i \leftarrow 0$  to 255 do
     $j \leftarrow (j + S[i] + k[j \bmod L]) \bmod 256$ 
    swap( $S[i], S[j]$ )
  end for
  return  $S$ 
end procedure
```

Algorithm : Dynamic Substitution Layer

This layer introduces an additional level of confusion by dynamically changing the s-boxes based on the number of iterations

```
procedure SUBSTITUTION( $\{Sbox_1, Sbox_2, Sbox_3\}$ )
  for  $i \leftarrow 0$  to  $nb$  do
    Encrypt(Block [ $i$ ])
     $it_1 \leftarrow it_1 + 1$ 
     $it_2 \leftarrow it_2 + 1$ 
    if  $it_1 = 2000$  then
       $Sbox_1 \leftarrow Sbox_2[Sbox_1]$ 
       $it_1 \leftarrow 0$ 
      if  $it_2 = 2000 \times 2000$  then
         $Sbox_2 \leftarrow Sbox_3[Sbox_2]$ 
         $it_2 \leftarrow 0$ 
      end if
    end if
  end for
end procedure
```

*Implemented along with
Encryption Module*

MODULE 3

Key Expansion Module

Key Expansion is used to derive round keys from a master key for each round of the Encryption process. It generates a series of round keys “ k_i ” and “ l_i ” from an initial master key. The process is based on the Feistel structure and uses modular addition, rotation, and XOR operations.

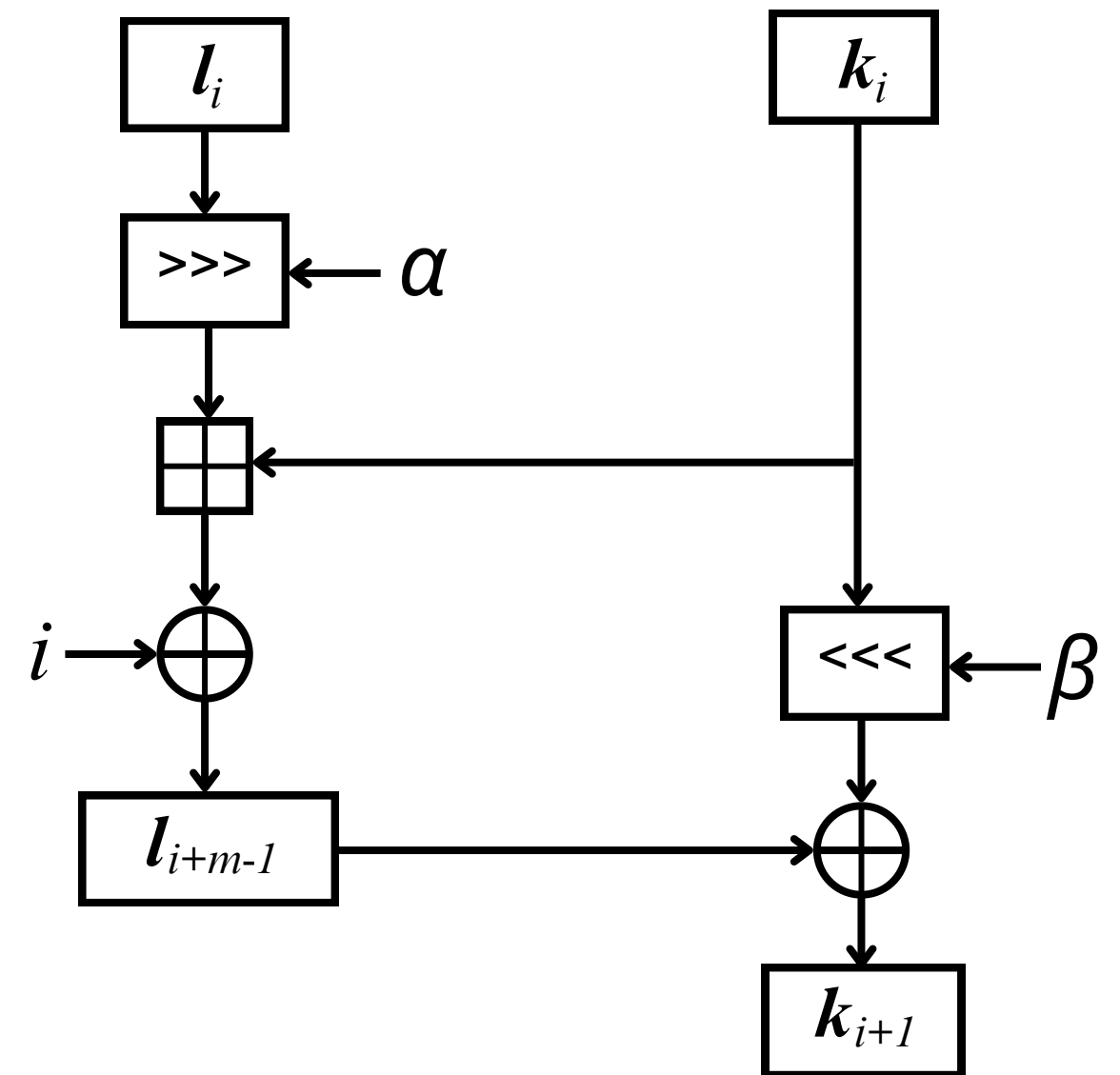
Parameters

- **m** : Number of Words in the Master Key. [$m = 3$] because the Key Size is 96 bits.
- **i** : Round Index
- **α & β** : Rotation Parameters. [$\alpha = 8, \beta = 3$]
- **l_i & k_i** : Intermediate values generated during the key expansion process

$$l_{i+m-1} = ((k_i \boxplus (l_i \gg \alpha)) \oplus i)$$

$$k_{i+1} = (k_i \ll \beta) \oplus l_{i+m-1}.$$

Initialization : $l_{m-2}, \dots, l_0, k_0$



MODULE 4

Encryption/Decryption Module

The encryption process in Speck-R is based on the ARX (Addition-Rotation-XOR) architecture with an added dynamic substitution layer. It operates in Counter (CTR) mode, allowing for parallel processing and efficient encryption/decryption.

Nonce Encryption

$$X_L = ((N_L \ggg 8) \boxplus N_R) \oplus K_r$$

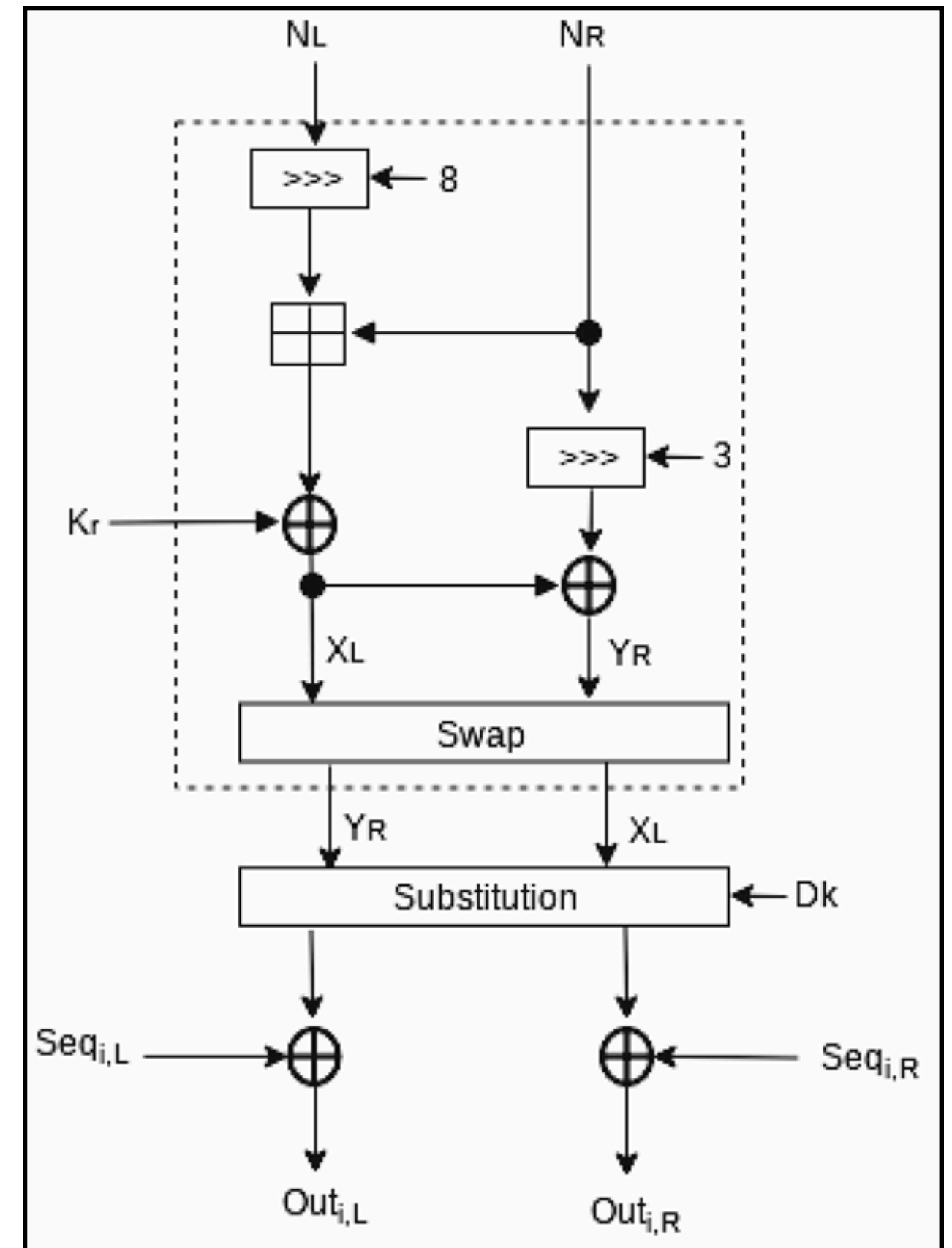
$$Y_R = (N_R \lll 3) \oplus X_L.$$

Substitution Layer

The Encrypted Nonce passes through a Dynamic Substitution Layer, which applies the Sboxes generated by the Dynamic Key.

XOR with PlainText

The substituted value is XORed with PlainText to produce CipherText



MODULE 4 (ongoing)

Encryption/Decryption Module

procedure SPECK-R_ENCRYPTION(Seq)

$N \leftarrow \text{PRNG}(\text{seed})$

$K \leftarrow \text{PRNG}(\text{seed})$

$K_r \leftarrow \text{Key Expansion}$

$DK \leftarrow \text{PRNG}(\text{seed})$

$\text{Seq}[nb] \leftarrow \text{plaintext}$

$Sbox_1, Sbox_2, Sbox_3 \leftarrow \text{RC4} - \text{KSA Initialization Algorithm}$

for $i \leftarrow 0$ to nb **do**

$X_L = ((N_L \ggg 8) \boxplus N_R) \oplus K_r$

$Y_R = (N_R \lll 3) \oplus X_L$

$\text{Swap}(X_L, Y_R)$

$N_L \leftarrow N_L$

$N_R \leftarrow N_R ++$

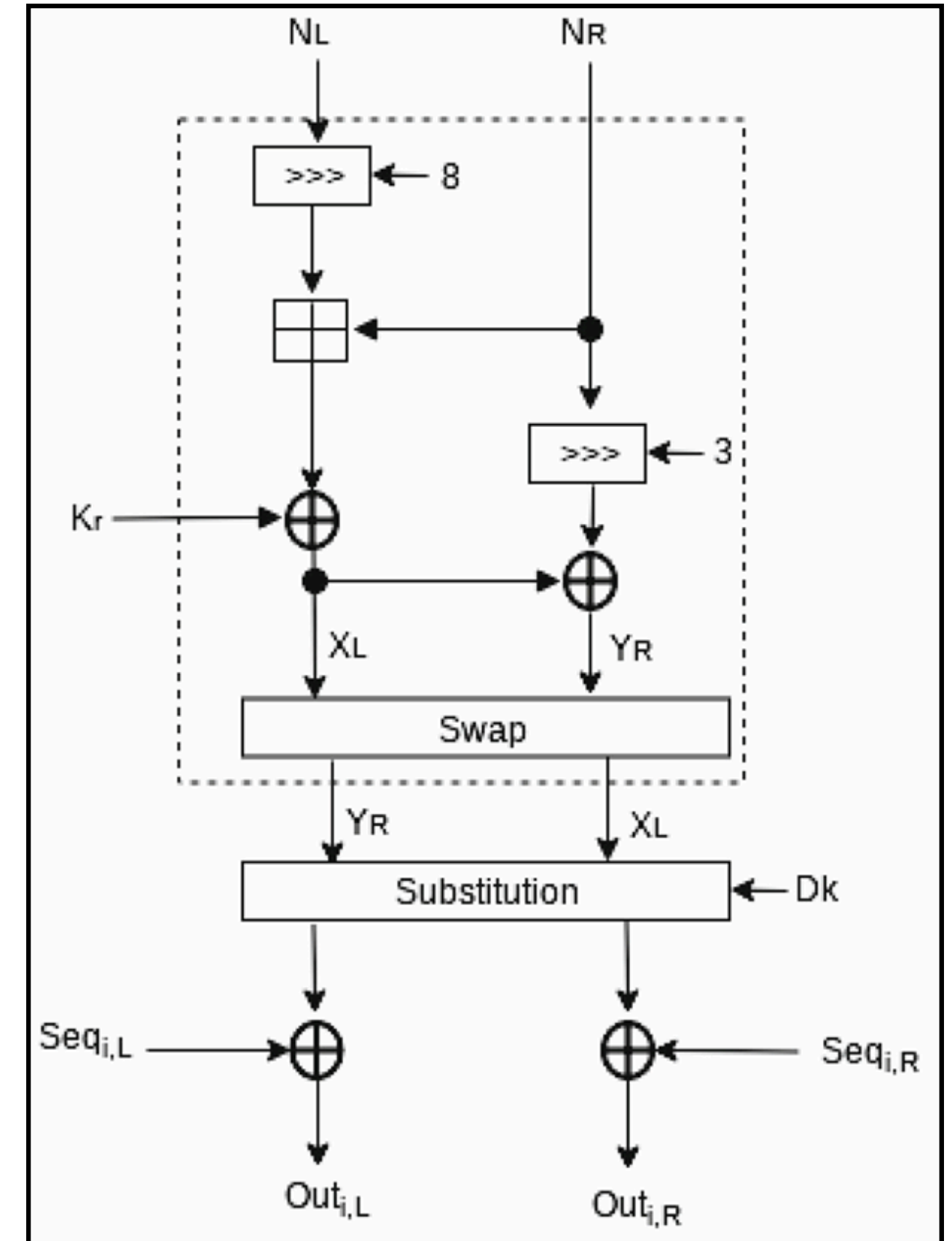
$\text{Out}_{i,L} \leftarrow Sbox_1[Y_R] \oplus \text{Seq}[i],L$

$\text{Out}_{i,R} \leftarrow Sbox_1[X_L] \oplus \text{Seq}[i],R$

$\text{Update } Sbox_1 \leftarrow \text{Substitution}(Sbox_1, Sbox_2, Sbox_3) \text{ Algorithm}$

end for

end procedure



RESULTS

The implementation of PRNG, N, K, DK generation, and key expansion for Speck-R has been done successfully.

- The PRNG generates 35 values from which N(64 bits), K(96 bits), and DK(256 bytes) are generated.
- The Master Key is Expanded for 7 rounds.

Speck-R Key Expansion:

$K_r[0] = 0x34642E66$

$K_r[1] = 0x4FD11215$

$K_r[2] = 0x0492C9DD$

$K_r[3] = 0xCDEC30B3$

$K_r[4] = 0x35E83F35$

$K_r[5] = 0x2D1C6563$

$K_r[6] = 0xABDD6647$

N (64-bit): 0x624F49FDA74F1E59
K (96-bit): 0xF4252CF57354F96934642E66
DK (256-byte):

27	99	B9	82	76	55	59	45	09	07	33	35	53	62	BD	74
03	0B	DD	27	C8	87	3B	6B	1E	DD	30	73	35	C4	20	E4
40	49	CA	10	82	34	93	09	80	AC	C4	4C	BB	A6	23	C5
98	B6	BE	B4	6B	CE	3D	9F	3B	8C	A6	1D	44	81	EB	20
98	5D	FE	D1	1A	DB	A6	B8	A4	72	B6	D3	27	3A	0B	F4
24	6A	5B	77	1C	19	CB	32	76	63	A8	33	7A	9E	8B	83
5F	88	4A	09	1E	62	79	A1	C6	01	7B	32	39	6B	52	D9
01	20	37	2E	58	FE	11	50	9C	C4	60	0F	3B	99	FF	26
A5	5B	18	24	22	ED	07	86	DE	AE	F2	07	49	EC	99	84
03	21	84	D2	FE	CF	77	7B	E5	55	8D	44	29	E3	51	23
65	0D	5B	6F	A6	89	86	72	CC	7E	F9	1C	DC	10	D6	1D
1A	C9	5A	E4	D1	08	3E	60	16	E2	18	4D	7E	E1	7A	81
A5	9A	C5	12	BD	D7	6F	7E	74	89	FB	C8	23	AF	BB	8D
DC	BE	72	1D	05	01	10	28	CB	AD	FF	E0	98	6A	97	45
EA	8F	A4	B4	52	2D	F7	99	9D	87	2A	C0	C9	3C	08	F7
26	E4	A8	FD	6C	98	83	61	5B	29	13	CC	BC	FE	31	D1

0x624F49FDA74F1E59
0xF4252CF57354F969
0xF44D1BC934642E66
0x4559557682B99927
0x74BD625335330709
0x6B3B87C827DD0B03
0xE420C4357330DD1E
0x993348210CA4940
0xC523A6BB4CC4AC80
0x9F3DCE6BB4BEB698
0x20EB81441DA68C3B
0xB8A6DB1AD1FE5D98
0xF40B3A27D3B672A4
0x32CB191C775B6A24
0x838B9E7A33A86376
0xA179621E094A885F
0xD9526B39327B01C6
0x5011FE582E372001
0x26FF993B0F60C49C
0x8607ED2224185BA5
0x8499EC4907F2AEDE
0x7B77CFFED2842103
0x2351E329448D55E5
0x728689A66F5B0D65
0x1DD610DC1CF97ECC
0x603E08D1E45AC91A
0x817AE17E4D18E216
0x7E6FD7BD12C59AA5
0x8DBBAF23C8FB8974
0x281001051D72BEDC
0x45976A98E0FFADCB
0x99F72D52B4A48FEA
0xF7083CC9C02A879D
0x6183986CFDA8E426
0xD131FEBCCC13295B

RESULTS

The S-BOX1, S-BOX2, and S-BOX3 has been generated successfully using KSA-RC4 Algorithm from the generated Dynamic Key (DK).

Sbox1:

E5	3B	7C	63	B6	D5	AF	80	04	19	A5	1E	7D	EC	B7	77
35	36	58	2A	39	DE	07	DD	92	8F	0F	E1	9D	22	3E	66
88	42	7A	86	5C	5D	55	9A	40	60	12	B8	26	00	B1	54
81	8B	6D	5B	E4	A0	69	A8	B4	44	F6	4E	68	73	9E	02
1C	E0	BA	7F	85	D3	3C	0B	DF	9F	32	5F	03	67	2B	71
C7	94	17	20	43	37	BD	D4	EA	70	56	B3	3D	8A	F7	9C
65	CC	E7	6C	A6	D2	D9	24	75	90	89	82	48	BC	C1	E6
83	05	09	11	49	72	B0	DC	EB	62	78	5E	01	0E	10	E8
33	B2	A4	FD	7B	13	79	23	AE	84	8C	21	99	DA	1F	25
28	EE	0A	98	93	CE	1B	E3	F9	2C	3F	4C	D1	4A	38	61
D0	F8	CD	F2	31	A2	FA	A1	AB	4D	FE	59	F4	08	97	34
B9	CA	87	A9	C9	1D	6A	3A	E9	A7	C3	6B	CB	96	AA	DB
5A	C6	46	2D	27	53	C8	C0	6F	D7	A3	F3	14	15	16	D8
F1	0D	FB	EF	FF	74	76	BE	D6	BF	C5	2E	29	52	6E	4F
45	4B	50	8D	8E	C4	57	51	06	9B	E2	ED	BB	7E	18	30
2F	47	41	F5	C2	AD	FC	91	B5	AC	CF	0C	1A	64	95	F0

Sbox2:

AA	81	56	89	0B	28	8B	3C	54	34	DC	26	21	AD	1B	11
55	DB	8F	77	32	79	D7	1E	71	A5	2D	88	63	8A	69	49
D5	FA	F8	2B	07	3F	E3	E0	31	B9	95	85	AC	4A	D0	4E
8C	91	51	97	A3	A6	68	04	62	14	C9	5F	DF	6F	D3	2A
65	E9	52	72	22	70	EC	F3	25	61	75	2F	CC	BE	7A	16
43	45	A7	B6	59	E6	02	C6	67	00	47	9E	7D	DD	A4	0A
B5	64	08	7B	3B	41	E8	24	C3	F5	C0	82	83	78	5A	9D
7E	E4	84	05	3A	7F	FD	1A	18	7C	A8	0C	87	D4	CD	BC
DA	92	09	A9	4F	50	F1	B1	CE	17	30	E7	C8	8D	0E	C2
9B	73	6C	FB	B4	90	F0	EF	99	12	1F	A0	B8	D2	5D	C1
F2	2E	35	FE	1D	80	F4	20	9C	B7	40	2C	A1	01	5C	CF
A2	3E	94	4D	5B	9F	42	03	C4	06	D8	D6	BB	BD	60	4C
BF	E1	46	E5	AB	6A	15	4B	6E	F9	0D	93	EB	B3	6B	9A
66	F6	5E	FC	53	E2	29	AF	ED	D9	36	1C	FF	C7	F7	74
39	6D	B0	33	27	EE	EA	B2	DE	37	96	CB	98	19	C5	8E
44	58	10	38	BA	3D	76	13	86	D1	CA	0F	57	48	23	AE

Sbox3:

98	F6	3C	4E	E8	C8	EE	33	DF	2E	1A	54	2B	79	2C	8E
8A	32	B7	34	40	46	73	7D	6C	C6	20	59	55	42	C0	D4
0D	93	52	AC	EA	A9	28	69	61	26	DC	BB	8B	75	AE	3E
24	DB	86	CB	FA	48	99	FD	80	29	89	97	3F	5B	3B	9C
67	37	58	1F	14	A8	4F	AB	35	F0	BE	2A	D9	12	CA	A4
BA	57	B8	EC	1C	03	EB	C4	5E	FE	8F	B1	F8	43	EF	2F
A1	83	94	47	51	88	00	64	7A	95	0E	85	8C	10	C5	16
B3	9F	08	3A	2D	0C	39	07	AD	C3	BD	E0	44	38	13	9D
F1	8D	E4	B6	E3	5D	4D	F3	7F	21	D8	B9	4A	66	9E	18
78	23	1D	7B	5F	0A	90	F4	DE	49	3D	FC	76	FF	4C	B4
04	15	BC	01	56	F5	19	A0	1B	E1	C9	6A	FB	9A	09	11
87	5A	CF	A5	B0	6E	7E	AA	F9	45	74	E6	7C	D2	53	4B
6F	D7	E5	71	C7	F7	84	06	BF	CE	5C	02	E9	36	DA	DD
E2	50	A7	D1	62	91	A2	C2	C1	68	27	6D	1E	65	E7	96
25	CD	D5	31	72	CC	D0	77	D3	05	81	70	AF	82	6B	0B
30	B5	D6	22	0F	41	17	60	F2	ED	9B	A3	63	B2	A6	92

RESULTS

Wireless connection between two ESP32s have been setup and data driven from a MAX30100 pulse oximeter sensor is securely transmitted over the wireless connection using SPECK-R cryptographic algorithm.

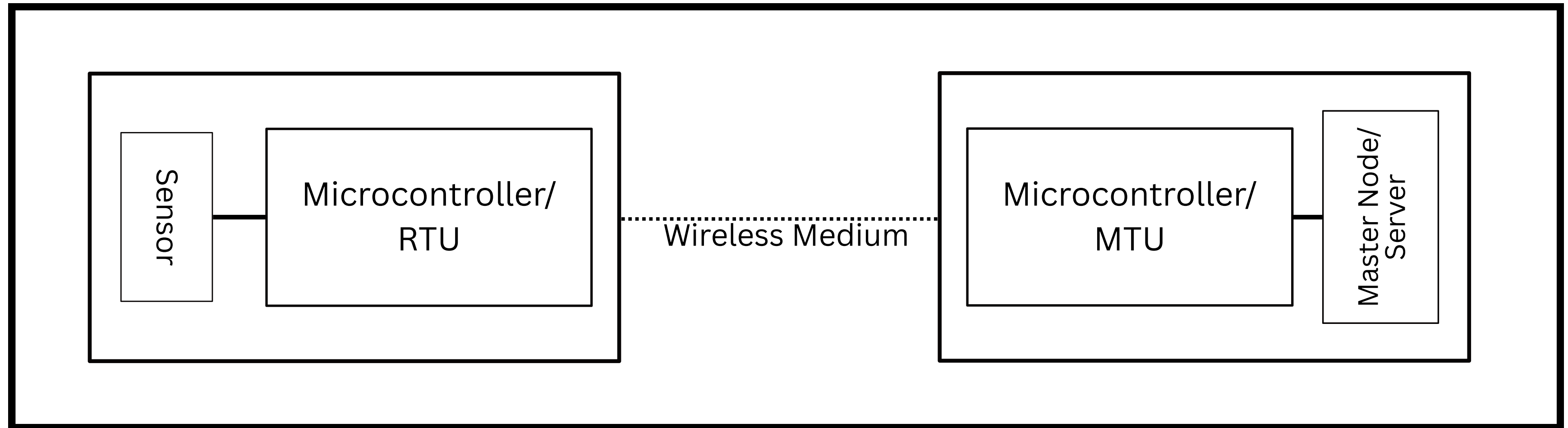
```
Server IP address (AP): 192.168.4.1
Server started.
AP Ready. Waiting for encrypted messages...
📡 Received Pulse Oximeter Reading:
❤️ Heart Rate: 88.81 bpm
🫁 SpO2: 95.00 %
-----
```

Server Output

```
...
Connected to AP!
TCP connection established...
Ready to auto-send encrypted HR/SpO2 every minute.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Getting sensor reading in 10 seconds...
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Beat detected.
Reading: HR = 88.81 bpm | SpO2 = 95.00 %
```

Client Output

Experimental Set-up



BLOCK DIAGRAM OF EXPERIMENTAL SETUP

CONCLUSION

- The proposed solution addresses the security challenges of resource-constrained IoT environments by implementing the ultra-lightweight SPECK-R cryptographic algorithm.
- By optimizing encryption for minimal computational overhead, power consumption, and latency, this approach ensures secure and efficient data transmission in applications like healthcare, industrial automation, smart homes, and wearable devices.
- The algorithm passes statistical tests to ensure cryptographic robustness and performance tests to validate its efficiency, making it a reliable solution for IoT security.

REFERENCES

1. Radhakrishnan, I.; Jadon, S.; Honnavalli, P.B. Efficiency and Security Evaluation of Lightweight Cryptographic Algorithms for Resource-Constrained IoT Devices. *Sensors* 2024, 24, 4008.
2. Sleem, L., Couturier, R. Speck-R: An ultra light-weight cryptographic scheme for Internet of Things. *Multimed Tools Appl* 80, 17067–17102 (2021).
3. L. Ertaul and A. Chauhan, "IoT Security: Implementation of Xtea, Simon/Speck Lightweight Block Ciphers," 2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE), Las Vegas, NV, USA, 2023, pp. 2478-2485.
4. Guy L. Steele Jr., Doug Lea, Christine H. Flood, Fast splittable pseudorandom number generators, *ACM SIGPLAN Notices*, Volume 49, Issue 10, Pages 453 - 472.
5. Q. Yu and C. N. Zhang, "RC4 state and its applications," 2011 Ninth Annual International Conference on Privacy, Security and Trust, Montreal, QC, Canada, 2011, pp. 264-269.
6. Kerry A. McKay, Larry Basshaz, Meltem Sönmez Turan, Nicky Mouha "Report on Lightweight Cryptography", National Institute of Standards and Technology Internal Report 81141 29 pages (August 2017)

THANK YOU